

# 离散与连续环境下的智能体决策系统：基于 DQN 与行为树的坦克 AI 研究

黄鹤翔 高恒斌 都奕宁

信息与电子工程学院

浙江大学, 杭州, 中国

**摘要**—本报告详细阐述了一个基于深度强化学习 (DQN) 与启发式决策的迷宫自动寻路及对战 AI 系统的设计与实现。项目的核心挑战在于不依赖现有的深度学习框架, 而是仅使用 NumPy 库从底层构建卷积神经网络及优化器。报告首先介绍了迷宫生成算法 (DFS、Prim) 及经典启发式寻路算法 (A\*), 随后深入探讨了自定义深度学习框架 simple\_nn 的实现细节, 包括 im2col 卷积加速、反向传播算法及 Adam 优化器。在此基础上, 搭建了 DQN 模型, 设计了包含墙壁信息、位置信息及访问记录的 7 通道状态空间, 并通过经验回放和目标网络机制稳定训练过程。同时, 探索了基于遗传算法的神经网络进化策略, 通过锦标赛选择与精英保留机制优化智能体决策。此外, 针对连续物理环境下的复杂对战需求, 设计了基于分层行为树的智能决策系统。该系统引入了基于物理引擎的弹道预测与反弹攻击策略、结合 Dijkstra 算法的全局路径规划及基于动作采样的动态规避算法, 有效解决了连续空间下的生存与精准打击难题。最后, 展示了 AI 在迷宫寻路测试及人机对战模式中的表现, 验证了算法的有效性。

**Index Terms**—深度强化学习, DQN, 卷积神经网络, NumPy, 迷宫生成, A\* 算法, 自动寻路, 启发式决策, 行为树, Dijkstra 算法, 弹道预测, 动态规避

## I. 引言

项目的设计灵感源于 B 站 up 河南胡季果和 L\_Shy\_P 的视频, 视频中出现了一个近乎无敌的 AI, 能够追踪敌人躲避子弹。

本项目旨在构建一个具备自主学习能力的坦克对战 AI 系统。与常规项目不同, 本项目的核心要求是“去框架化”, 即不调用成熟的深度学习库, 而是利用 Python 和 NumPy 从零实现神经网络的底层算子及训练机制。同时, 本项目还尝试了遗传算法作为另一种优化手段, 通过模拟生物进化过程来迭代神经网络参数。

为了应对更复杂的连续物理环境对战场景, 本项目还设计了一套基于启发式规则与行为树的智能决策系

统。传统的 DQN 模型虽然在离散网格中表现优异, 但在处理连续坐标、任意角度旋转及精细碰撞检测时效果很糟糕。所以本项目构建了 Smart AI 架构, 结合物理弹道推演、网格路径搜索及预测性闪避策略进行决策。这种启发式决策的设计保证了连续环境下的高水平竞技表现。

## II. 设计任务和要求

本项目的具体设计任务和要求如下:

### 1) 迷宫环境构建:

- 实现基于深度优先搜索和随机 Prim 算法的迷宫生成器, 支持生成不同拓扑结构的地图。
- 实现 A\* 启发式寻路算法, 作为 AI 路径规划的基准。

### 2) 底层深度学习框架实现:

- 仅使用 NumPy 库, 从零构建神经网络核心算子, 包括全连接层、卷积层及激活函数。
- 实现 im2col 技术以加速卷积运算, 并推导实现各层的反向传播梯度计算。
- 实现 Adam 自适应优化器及 MSE Loss 损失函数, 支持网络的参数更新。

### 3) 离散环境强化学习 AI:

- 构建基于 CNN 的 Q 值估计网络, 设计包含墙壁、位置及轨迹信息的 7 通道状态空间。
- 实现经验回放与目标网络机制, 解决训练不稳定的问题。
- 在离散网格环境中训练智能体, 实现从随机探索到最优路径规划的自主学习。

### 4) 遗传算法进化策略:

- 构建全连接神经网络作为决策模型，探索非梯度下降类的优化方法。
- 实现基于锦标赛选择、均匀交叉和高斯变异的遗传算法，用于神经网络参数的进化迭代。

#### 5) 连续环境启发式智能 AI:

- **危险感知**: 设计基于物理引擎的弹道预测机制，模拟子弹反弹与碰撞，识别潜在威胁。
- **决策系统**: 构建分层行为树，实现生存优先、动态规避、反弹攻击与路径巡逻的状态切换。
- **运动控制**: 设计“网格 Dijkstra + 局部 PID 控制”的分层路径规划算法，并实现防卡死摆脱逻辑。

#### 6) 人机对战系统集成: 集成上述模型，实现流畅的实时对战功能，并提供 AI 决策的可视化调试。

### III. 算法原理

#### A. 迷宫生成算法-DFS

深度优先搜索算法 (DFS) 是一种用于遍历或搜索树或图的算法。这个算法会尽可能深地搜索树的分支。当节点  $v$  的所在边都已被探寻过，搜索将回溯到发现节点  $v$  的那条边的起始节点。这一过程一直进行到已发现从源节点可达的所有节点为止。如果还存在未被发现的节点，则选择其中一个作为源节点并重复以上过程，整个进程反复进行直到所有节点都被访问为止。

在迷宫生成中，DFS 算法通过以下步骤实现：

- 将每个网格单元视为图中的一个节点；
- 初始时所有节点标记为“未访问”，所有相邻节点间存在“墙”；
- 从任意起点（如左上角  $(0,0)$ ）开始 DFS 遍历；
- 每次移动到新节点时，移除当前节点与目标节点之间的墙，并将目标节点加入连通图；
- 递归完成后，整个网格形成一棵生成树，对应一个无环连通迷宫。

#### B. 迷宫生成算法-prim

prim 算法是一种贪心算法，可在一个加权连通图中找到其最小生成树。此算法搜索到的边子集所构成的树中，不但包括了连通图里的所有顶点，且其所有边的权值之和亦为最小。

具体步骤如下：从单一顶点开始，普里姆算法按照以下步骤逐步扩大树中所含顶点的数目，直到遍及连通图的所有顶点

- 1) 输入：一个加权连通图，其中顶点集合为  $V$ ，边集合为  $E$ ；
- 2) 初始化：  $V_{\text{new}} = \{x\}$ ，其中  $x$  为集合  $V$  中的任一节点（起始点），  $E_{\text{new}} = \{\}$ ；
- 3) 重复下列操作，直到  $V_{\text{new}} = V$ ：
  - a) 在集合  $E$  中选取权值最小的边  $(u, v)$ ，其中  $u$  为集合  $V_{\text{new}}$  中的元素，而  $v$  是  $V$  中尚未加入  $V_{\text{new}}$  的顶点（如果存在有多条满足条件且具有相同权值的边，可任意选取其中之一）；
  - b) 将  $v$  加入集合  $V_{\text{new}}$  中，将  $(u, v)$  加入集合  $E_{\text{new}}$  中；
- 4) 输出：使用集合  $V_{\text{new}}$  和  $E_{\text{new}}$  来描述所得到的最小生成树。

在迷宫生成中具体应用如下：

- 将网格视为完全图，每条潜在通道（相邻格子间）是一条边；
- 任选一个起始格子加入生成树；
- 将其所有相邻格子加入“边界集”；
- 从边界集中随机选一格子，随机选择其一个已在生成树中的邻居，打通两者之间的墙，并将该格子纳入生成树，同时将其未访问邻居加入边界集；
- 直至所有格子被纳入。

DFS 生成的迷宫通常具有长而曲折的走廊和较多的分支，Prim 生成的迷宫通常具有更多短分支和较短的主路径。

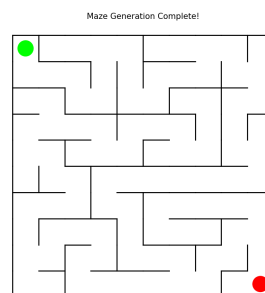


图 1: DFS 生成迷宫

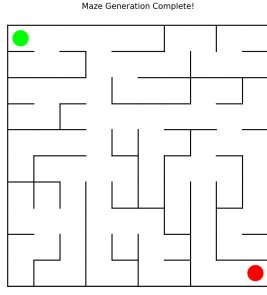


图 2: prim 生成迷宫

### C. 解迷宫算法-A\*

A\* 算法是一种广泛应用于路径规划和图遍历的启发式搜索算法。它结合了 Dijkstra 算法的完备性与贪心最佳优先搜索的效率，能够高效地找到从起点到终点的最短路径。

A\* 算法通过评估函数  $f(n)$  来决定下一个要探索的节点：

$$f(n) = g(n) + h(n)$$

其中：

- $g(n)$ : 从起点到当前节点  $n$  的实际代价。
- $h(n)$ : 从当前节点  $n$  到目标点的启发式估计代价。

在本项目中使用曼哈顿距离  $\|x_1 - x_2\| + \|y_1 - y_2\|$  进行启发式搜索。

- 1) 在初始化时将起点 (0,0) 加入 openset 中，然后进入循环，直到 openset 为空或找到终点为止。
- 2) 在每一步迭代中，从 open 列表中选择  $f$  值最小的节点作为当前节点
- 3) 检查当前节点是否为目标节点，如果是则结束循环；如果不是目标节点，则将其标记为已处理并将其相邻的未处理节点加入 open 列表中，同时将已处理的节点加入 closedset 中以避免重复处理。

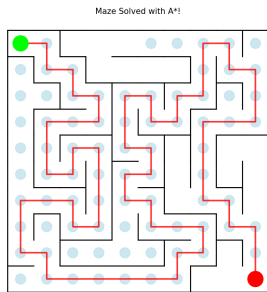


图 3: A\* 算法解迷宫

### D. 自定义深度学习框架 (simple\_nn)

为了替代 PyTorch，我们实现了 `simple_nn.py`。

1) 卷积层实现：卷积操作的核心在于高效地提取局部特征。为了避免低效的多重循环，采用了 `im2col` 技术，将输入特征图展开为矩阵，从而将卷积运算转化为矩阵乘法。

$$Y = W_{col} \times X_{col} + b \quad (1)$$

其中  $X_{col}$  是展开后的输入矩阵， $W_{col}$  是重排后的权重矩阵。

2) 优化器：实现了 Adam 优化器，结合了动量法和 RMSProp 的优点，自适应调整学习率。

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (2)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (3)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t \quad (4)$$

### E. AI 训练-DQN

DQN (Deep Q-Network) 将卷积神经网络与 Q-Learning 结合，解决了高维状态空间的强化学习问题。

1) 核心机制：

- 经验回放：构建一个回放池  $D$ ，存储转移样本  $(s, a, r, s', done)$ 。训练时随机采样一个小批量，打破了数据间的相关性，提高了训练的稳定性。
- 目标网络：引入一个参数滞后的目标网络  $\hat{Q}$  计算目标值，避免了“自举”导致的目标值震荡。目标网络每一定时间从策略网络中复制参数。

通过贝尔曼最优方程来计算期望  $Q$  值，即执行该动作后获得的即时奖励  $r$  加上对未来最大  $Q$  值的折扣期望

$$Q^*(s, a) = \mathbf{E}_{s' \sim \mathcal{P}} [r + \gamma \max_{a'} Q^*(s', a')]$$

目标函数为最小化均方误差：

$$L(\theta) = \mathbf{E} \left[ \left( r + \gamma \max_{a'} \hat{Q}(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (5)$$

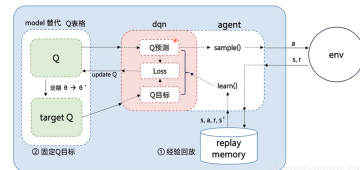


图 4: dqn 算法流程结构

2)  $Q$  值估计网络设计-输入层: 输入层设计为  $7 \times H \times W$  的张量, 包含 7 个通道:

- 通道 0-3 (墙壁信息): 分别编码每个格子上、下、左、右四个方向的墙壁存在情况
- 通道 4-5 (位置信息): 分别使用 One-hot 编码智能体位置和目标位置
- 通道 6 (历史轨迹): 记录智能体访问过的路径, 赋予智能体记忆

3)  $Q$  值估计网络设计-卷积层: 网络包含 4 层卷积层, 通道数分别为 32, 64, 128, 128, 主要为卷积层与 relu 激活层的循环。网络中没有加入传统 CNN 网络中常见的池化层, 主要是因为池化层的加入会模糊智能体与目标点的位置信息, 影响训练效果。

通过堆叠 4 层卷积层, 深层神经元的感受野逐渐扩大, 能够感知更大范围的迷宫结构, 从而规划长距离路径。通道数逐层增加 ( $32 \rightarrow 128$ ), 使得网络能够组合低级几何特征。

表 I: 卷积神经网络各层参数配置

|     | 输入通道数 | 输出通道数 | 卷积核大小 | 步长 | 填充 |
|-----|-------|-------|-------|----|----|
| 第一层 | 7     | 32    | 3     | 1  | 1  |
| 第二层 | 32    | 64    | 3     | 1  | 1  |
| 第三层 | 64    | 128   | 3     | 1  | 1  |
| 第四层 | 128   | 128   | 3     | 1  | 1  |

4)  $Q$  值估计网络设计-全连接层: 卷积层输出展平后, 连接 3 层全连接层 ( $1024 \rightarrow 512 \rightarrow 4$ ), 提供了强大的非线性拟合能力, 将提取的迷宫特征转化为上、下、左、右四个离散动作的  $Q$  值。

#### F. 奖励设计

```
# 访问新格子的奖励
reward = 0.2

if self.agent_pos == self.goal_pos:
    # 到达目标点的奖励
    reward = 20.0
    done = True
else:
    # 撞墙惩罚
    reward = -1.0
```

#### G. 超参数设计

```
# Hyperparameters
WIDTH, HEIGHT = 10, 10
EPISODES = 10000
BATCH_SIZE = 32
GAMMA = 0.9 # 对未来奖励的关心程度, 越大越关心
# 使用线性下降的EPSILON, EPSILON越大越倾向于随机探索
EPSILON_START = 1.0
EPSILON_END = 0.05
EPSILON_DECAY_EPISODES = 4000
LR = 0.0001
TARGET_UPDATE = 200 # 目标网络的更新频率
MEMORY_SIZE = 50000 # 经验池的大小
```

#### H. AI 训练-遗传算法

遗传算法是一种受达尔文生物进化论启发的元启发式优化算法。它模拟了自然界“物竞天择, 适者生存”的过程, 通过在解空间中维护一个种群, 并利用选择、交叉和变异等遗传算子, 迭代地演化出越来越优的解决方案。

在这个项目中, 我们使用一个全连接神经网络来对智能体的下一步动作进行决策, 通过遗传算法迭代更新神经网络的参数。同时在训练过程中, 我们借鉴了 DQN 的训练思路, 每隔一段时间将训练目标的参数从智能体中复制下来, 以求获得更加强大的智能体。

- 选择算子: 锦标赛选择
- 交叉算子: 均匀交叉
- 变异算子: 高斯扰动
- 进化策略: 精英保留, 将每一代中适应度最高的前 10% 个体 (精英) 直接复制到下一代, 不经过交叉和变异

```
# DQN Agent
# 每一步惩罚使得ai寻找最短路径
reward = -0.1
done = False

if (nx, ny) in neighbors:
    self.agent_pos = (nx, ny)

    if self.visited_map[ny, nx] > 0:
        # 格子重复访问惩罚
        reward = -0.5
    else:
        self.visited_map[ny, nx] = 1.0
```

## I. 连续环境下的智能决策系统

随着游戏环境从离散网格升级为连续物理环境，原来的使用 DQN 训练的泛化能力有所不足，选择构建了基于决策树的智能体。新的智能体不再局限于上下左右的离散移动，而是基于连续的坐标体系和角度控制，实现了更加精准的战术动作。

1) 基于物理推演的威胁感知与规避：在连续环境中，子弹的轨迹不再是简单的网格跳跃。智能体引入了基于物理引擎的轨迹预测机制。系统每帧模拟场上所有子弹在未来时间窗口内的运动，包括位置更新与墙壁反弹计算。

$$P_{t+1} = P_t + V_t \cdot \Delta t, \quad V_{t+1} = \text{Reflect}(V_t, N_{\text{wall}})$$

通过检测预测轨迹线段与智能体碰撞盒（引入安全半径  $R_{\text{safe}} = 30\text{px}$ ）的相交情况，识别潜在威胁。

当检测到威胁时，系统采用基于离散动作空间的动态规避策略。系统预定义了包含 9 种基础动作的候选集  $\mathcal{A}$ ，并按照代码中定义的优先级顺序进行安全性评估：

- 1) **原地机动**：优先尝试停止或原地旋转（顺/逆时针），以最小的位移代价规避威胁。
- 2) **直线机动**：若原地机动无效，尝试沿当前朝向直线进退。
- 3) **复合机动**：最后尝试“边走边转”的曲线运动，以应对复杂的封锁弹幕。

决策过程分为两轮：第一轮按优先级遍历候选集，一旦发现某动作在未来时间窗口内无碰撞，立即执行该动作；若所有动作均存在风险，则进入第二轮评估，计算每个动作在模拟轨迹中与最近威胁的最小距离，选择不危险的动作：

$$a^* = \arg \max_{a \in \mathcal{A}} \left( \min_{b \in \mathcal{B}_{\text{threat}}} \text{dist}(S_{\text{player}}(a), \text{Traj}_b) \right) \quad (6)$$

2) 反弹攻击策略：攻击模块不仅检测直接视线，还利用射线投射技术模拟子弹反弹。智能体以一定角度步长扫描 360 度方向，寻找可以通过一次墙壁反弹击中敌人的射击解。若直接射击路径被墙壁阻挡，智能体会计算反弹路径：

$$\theta_{\text{aim}} = \text{FindRicochet}(\theta_{\text{scan}}) \quad \text{s.t.} \quad \text{Intersect}(\text{Ray}(\theta_{\text{scan}}), P_{\text{enemy}}) \quad (7)$$

这使得智能体具备了“隔山打牛”的高级战术能力，能够在掩体后方安全地攻击敌人。

3) 全局路径规划与运动控制：移动系统采用分层架构，将高层的离散路径规划与底层的连续运动控制解耦，实现了从网格决策到物理执行的平滑过渡。

a) 1. 离散空间的最短路径搜索：上层规划器首先将连续的物理世界离散化。根据地图尺寸 ( $C_{\text{cols}} \times C_{\text{rows}}$ )，将战场划分为网格单元。系统构建图  $G = (V, E)$ ，其中节点  $v \in V$  代表无障碍的网格中心。利用 **Dijkstra 算法** 计算从当前位置  $P_{\text{start}}$  到目标位置  $P_{\text{end}}$  的最优路径序列。路径点集合  $\mathcal{P}$  由一系列世界坐标组成：

$$\mathcal{P} = \{w_0, w_1, \dots, w_n\} \quad \text{where } w_i = \text{CenterOf}(\text{Cell}_i) \quad (8)$$

为了适应连续环境，路径点  $w_i$  被直接映射为物理引擎中的  $(x, y)$  坐标，而非简单的网格索引。

b) 2. 基于阈值的非线性追踪控制：下层控制器负责驱动坦克沿路径点  $\mathcal{P}$  移动。控制器实时计算当前位置  $P_{\text{curr}}$  与当前目标路点  $w_{\text{next}}$  之间的欧几里得距离  $d$ 。当  $d < 25\text{px}$  时，判定为到达该路点，目标切换至  $w_{\text{next}+1}$ 。

姿态控制采用基于角度误差的 **Bang-Bang 控制器**（开关控制），而非线性的比例控制。系统首先计算期望航向角  $\theta_{\text{des}}$  和当前航向角  $\theta_{\text{curr}}$  之间的归一化误差  $\delta$ ：

$$\theta_{\text{des}} = \text{atan2}(y_{\text{target}} - y_{\text{curr}}, x_{\text{target}} - x_{\text{curr}}) \quad (9)$$

$$\delta = \text{Normalize}(\theta_{\text{des}} - \theta_{\text{curr}}) \in (-\pi, \pi] \quad (10)$$

控制律设计了两个关键的阈值：转向阈值  $\epsilon_{\text{steer}} \approx 0.2 \text{ rad}$  ( $11.5^\circ$ ) 和移动阈值  $\epsilon_{\text{move}} \approx 0.78 \text{ rad}$  ( $45^\circ$ )。

• **转向控制**：当角度误差超过死区时，全速转向。

$$u_{\text{steer}} = \begin{cases} \text{sgn}(\delta) \cdot 1.0 & \text{if } |\delta| \geq \epsilon_{\text{steer}} \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

• **油门控制**：仅在车头大致对准目标时才施加动力，即大角度原地转向，小角度边走边转。

$$u_{\text{throttle}} = \begin{cases} 1.0 & \text{if } |\delta| < \epsilon_{\text{move}} \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

c) 3. 滞后检测与异常恢复：为了避免常见的墙角卡死问题，系统引入了基于位移的异常检测机制。



首先定义状态判定函数,当且仅当控制指令  $u_{throttle} > 0$  但实际位移  $\|\Delta P\| < 0.5px$  时,判定为“卡死”状态。恢复策略分为两个阶段:

- **阶段一 (轻微卡顿, 0-5 帧):** 强制输出转向指令 ( $u_{steer} = 1.0$ )。利用坦克的矩形碰撞盒在旋转时产生的物理挤压,尝试将车体“滑”出墙壁边缘。
  - **阶段二 (严重卡死, 大于 5 帧):** 若旋转无效,系统判定前方完全受阻,强制执行倒车机动 ( $u_{throttle} = -1.0, u_{steer} = 0$ ),直到脱离困境。
- 连续环境下整体场景如下图所示:

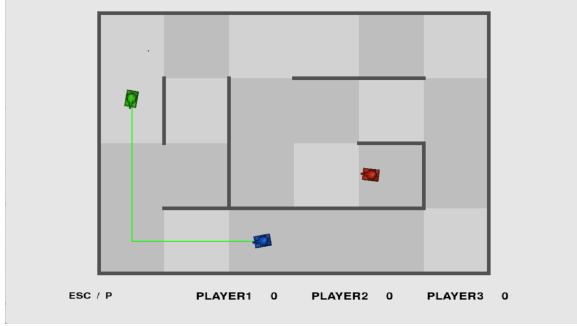


图 5: 连续环境下的坦克对战场景

#### IV. 主要仪器设备

- **开发语言:** Python 3.10
- **核心库:** NumPy (用于矩阵运算), Matplotlib (用于可视化)
- **硬件环境:** AMD CPU
- **操作系统:** Linux/Windows

#### V. 设计结果

对于简单环境的 DQN 训练, reward 曲线如下图所示

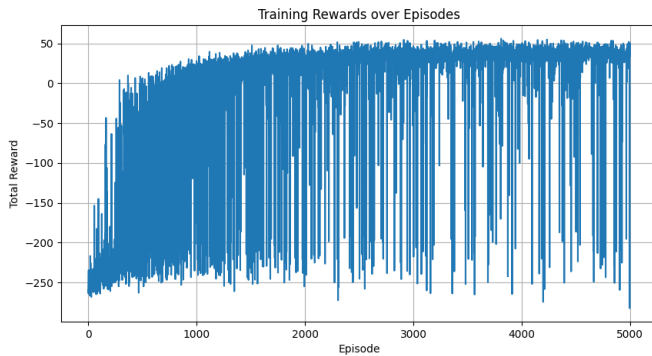


图 6: dqn 奖励变换曲线

更多结果请参见代码压缩包与视频压缩包

#### VI. 结论

本报告详细阐述了从零构建深度强化学习框架及坦克对战 AI 的全过程。我们首先成功实现了基于 NumPy 的底层神经网络框架,并通过 DQN 算法在离散迷宫中完成了初步的寻路训练,验证了“去框架化”深度学习的可行性。

同时,遗传算法的实验结果表明,在特定任务下,基于进化的参数搜索策略也能取得与梯度下降相媲美的效果,为强化学习提供了有益的补充。

随后,针对实战中复杂的连续物理环境,我们突破了离散动作空间的限制,开发了基于行为树、弹道预测及动态规避策略的 Smart AI。该系统成功解决了连续空间下的碰撞检测、动态规避及平滑控制等难题。实验结果表明,Smart AI 在动态规避子弹和战术进攻方面表现优异,展现了启发式算法在实时对抗类游戏中的鲁棒性与高效性。

本项目不仅加深了对深度学习底层原理的理解,也探索了将传统游戏 AI 技术与现代物理引擎结合的有效路径,为后续研究更复杂的强化学习算法奠定了坚实基础。

#### VII. 项目分工与产出

表 II: 人员分工

| 姓名  | 学号         | 分工                    | 贡献占比 |
|-----|------------|-----------------------|------|
| 黄鹤翔 | 3230106231 | 简单环境迷宫生成、决策 ai、DQN 训练 | 10   |
| 高恒斌 | 3230105132 | 连续环境决策 ai             | 10   |
| 都奕宁 | 3230100948 | 简单环境决策、连续环境决策         | 10   |

本项目的完整实现代码、训练脚本已开源在 GitHub 仓库: <https://github.com/GgbNnD/tankalg>

#### VIII. 参考文献

- 1) Mnih, V., Kavukcuoglu, K., Silver, D. et al. Human-level control through deep reinforcement learning. Nature 518, 529–533 (2015).
- 2) Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100-107.

- 3) Liang, Y. D., & Barsky, B. A. (1984). A new concept and method for Line Clipping. ACM Transactions on Graphics (TOG), 3(1), 1-22.
- 4) Holland, J. H. (1992). Adaptation in natural and artificial systems. MIT press.
- 5) 河南 胡季果, L\_Shy\_P. (2024). Bilibili Video: <https://www.bilibili.com/video/BV17EswzLEKw/>

## IX. 引用

连续环境下游戏引擎参考自 GitHub 开源项目: <https://github.com/mglyn/TANKTROUBLE-pythonedition.git>, 其依赖于 numpy/pygame 库